



PROJECT REPORT

빵디즈 프로젝트

최종 보고

빵빠레(3팀): 주병규(팀장) | 조형욱 | 노여진 | 김재성 | 손하영

목차



프로젝트 소개

주제, 문제 정의, 목표 및 핵심 기능, 개발 일정, 설계도 등 프로젝트의 전반적인 개요를 소개합니다.



협업 도구 & 인프라

MSA 아키텍처, 브로커 선택, CI/CD 파이프라인 및 모노레포 구조에 대해 설명합니다.



서비스별 설계

채팅, 인증·멤버, 주문, 결제, 알림 등 서비스별 도메인 설계와 핵심 기술 적용 사례를 소개합니다.



SECTION 01

프로젝트 소개

주제, 문제 정의, 목표 및 핵심 기능,
개발 일정 등 프로젝트의 전반적인 개요를 소개합니다.

주제 및 목표

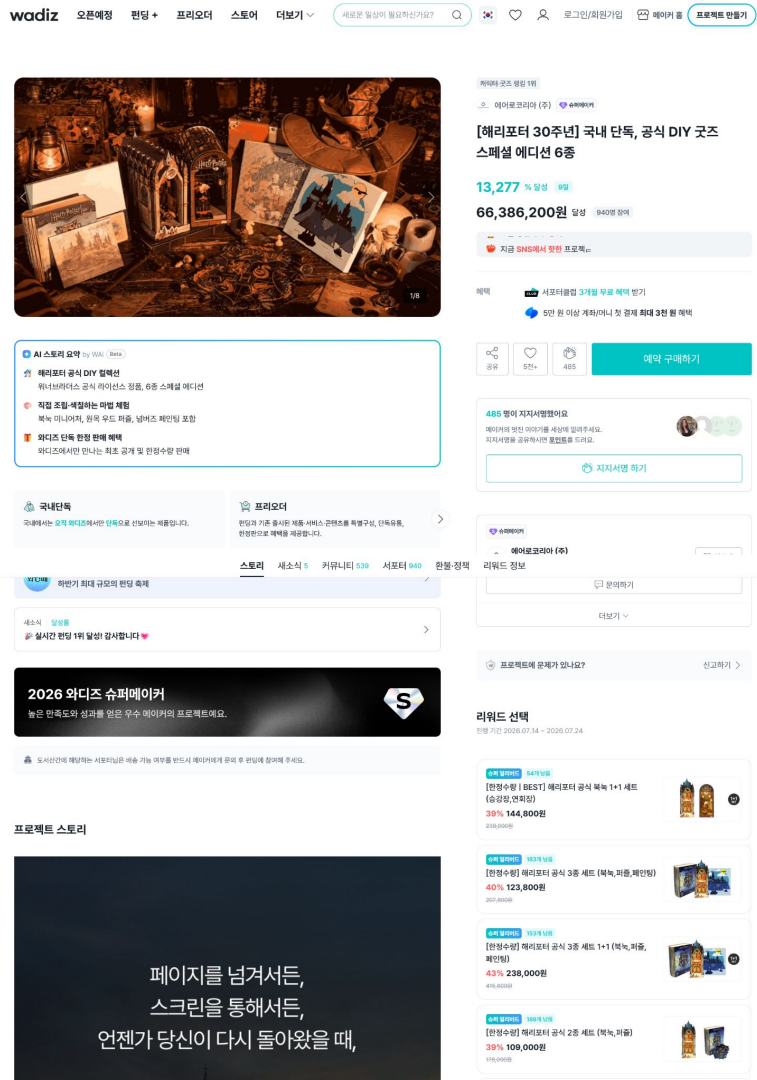
빵디즈는 사용자 간의 자유로운 펀딩을 지원하며, 펀딩 기간 동안 열리는 **실시간 공개방**을 통해 상품에 대한 토론과 소통을 이어가는 혁신적인 플랫폼입니다.



문제 정의

소통의 부재와 실패한 후원

기존 대형 펀딩 플랫폼은 후원자와 창작자, 또는 후원자 간의 **직접적인 소통** 수단이 매우 부족합니다.



해결 방안



실시간 소통을 통한 검증

빵디즈는 펀딩별 **공개 채팅방**을 도입하여 제품에 대한 심도 있는 토의를 가능하게 합니다.

이를 통해 후원자가 **"나에게 정말 필요한 상품인가"**를 객관적으로 판단할 기준점을 스스로 세우도록 돕습니다. 결과적으로 맹목적인 충동 후원을 방지하고, 펀딩의 질적 향상과 만족도를 높이는 경험을 제공합니다.

목표의 핵심 기능

커뮤니티형 펀딩

단순한 구매를 넘어, 관심사가 같은 사용자들이 모여 자유롭게 의견을 나누고 상품의 가치를 함께 발전시킵니다.

스마트 펀딩 시스템

누구나 쉽게 참여하고 진행 상황을 투명하게 확인할 수 있는 안정적인 펀딩 환경을 제공합니다.

실시간 알림 서비스

펀딩 성공 여부, 공개방의 주요 대화, 새로운 소식 등 놓치기 쉬운 핵심 정보를 실시간으로 신속하게 전달합니다.

자체 월렛 기반 결제 시스템

사이트 전용 월렛을 통해 결제하며, 펀딩 성공/실패 상황에 따라 자동 환불이 이루어져 안전한 자금 흐름을 보장합니다.

개발 일정





SECTION 02

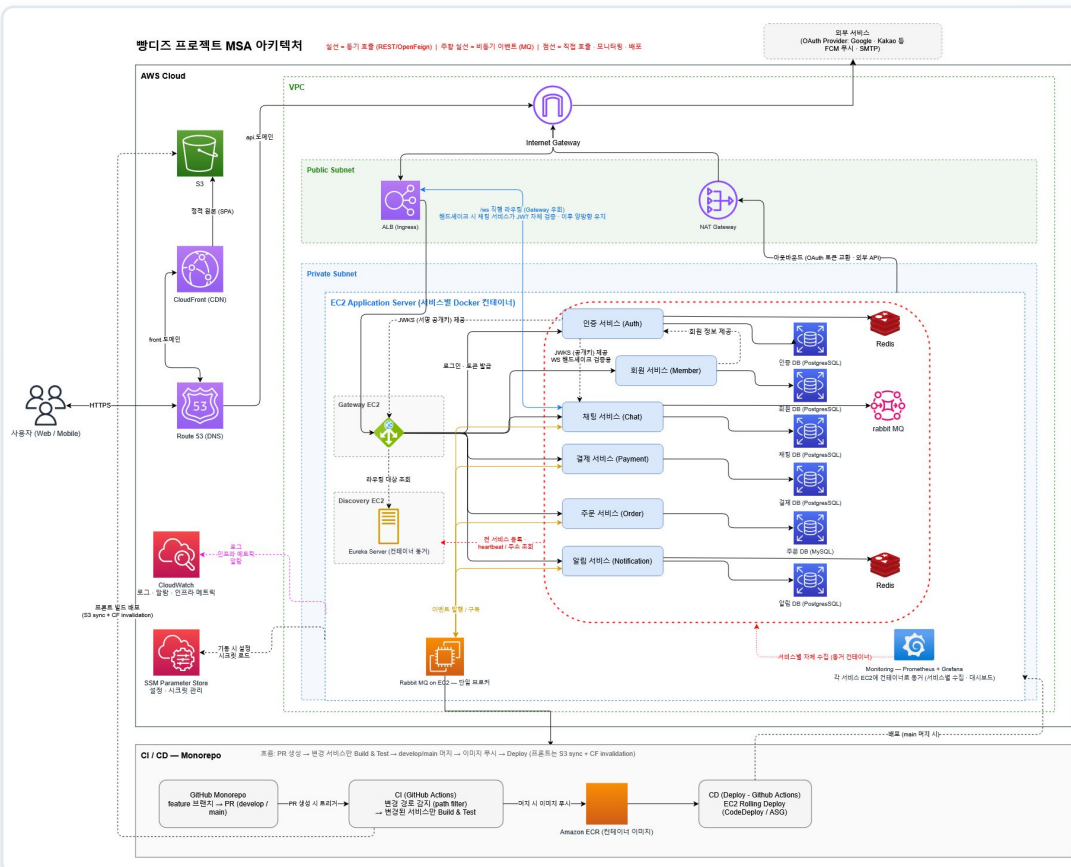
협업 도구 & 인프라

MSA 아키텍처, 브로커 선택(RabbitMQ),
CI/CD 파이프라인 및 모노레포 구조에 대해 설명합니다.

기술 스택 및 아키텍처

분류	상세 기술 스택
아키텍처 & 언어	MSA (Microservices Architecture), Java 25
프레임워크	Spring Boot 4.1.0, Spring Cloud 2025.1.2 (Oakwood)
인증 및 실시간	Spring Security, JWT, WebSocket, STOMP
MSA 통신 & 라우팅	RabbitMQ:4, Spring Cloud Gateway, Eureka (Service Discovery)
테스트 & 인프라	JUnit 5, Mockito, Testcontainers, Docker, Docker Compose
데이터베이스 & ORM	Postgresql:17, Mysql:8.4, Spring Data JPA, redis

기술 스택 및 아키텍처



수정 사항

인증 · 알림

Redis docker 추가

접근 방식

Spring Cloud Gateway와 Eureka 리소스 서버를 분리

배포 방식

EC2 Rolling Deploy (CodeDeploy / ASG)

→ SSM Run Command 방식으로 전환

(Docker pull & run, EC2 대당 1대 구성)

구성 요소별 재산정 및 월간 예산

항목	수량	단가	소계
EC2 t3.medium (auth·member·chat·payment·order·notification)	6	\$0.052	\$0.312
EC2 t3.small (gateway·discovery·rabbitmq)	3	\$0.026	\$0.078
Redis EC2 (t3.small 가정)	1	\$0.026	\$0.026
NAT Gateway	1	\$0.059	\$0.059
ALB (고정 + LCU)	1	\$0.0225+\$0.008	\$0.031
퍼블릭 IPv4 (NAT+ALB)	1	\$0.005	\$0.015
EBS gp3 (10대 × 30GB)	300GB	\$0.0912/GB·월	\$0.037
데이터 전송·ECR·CloudWatch 등 (실측 역산)			≈ \$0.07/시간
실측 기준 합계			\$0.628/시간

월간 예산 시나리오

24시간 상시 가동

\$163.8/월

\$5.46/일 × 30일
= 월 \$163.8
(약 226,000원, 환율 1,380원 기준)

평일 09~18시 · 22일 영업

\$58.6/월

176시간 × \$0.228 = \$40.1
544시간 × \$0.034 = \$18.5
합계 ≈ \$58.6/월 (약 81,000원)

완전 OFF

\$24.5/월

EBS + 호스팅존 기준
고정 비용만 발생

디렉터리 구조 (Monorepo)

📁 bds_backend/

├── 📁 .github/ // CI/CD 워크플로우, 이슈 템플릿

├── 📁 docs/ // API 스펙, 프로젝트 가이드

├── 📁 infra/ // RabbitMQ 로컬/dev 설정

├── 📁 libs/ // 서비스 간 공통 이벤트 타입

├── 📁 modules/ // 공통 유틸리티, 메시징 설정(RabbitMQ 공통 설정)

├── 📁 platform/ // Config / Discovery 서버

└── 📁 services/

├── auth-service

├── chat-service

├── member-service

└── ...기타 서비스

Layered Architecture

Presentation

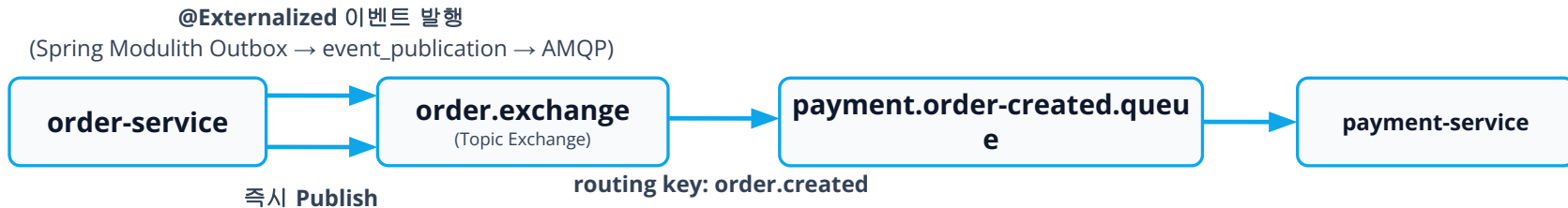
Application

Domain

Infra



서비스 간 통신 구조



Topic Exchange

서비스 간 메시지는 RabbitMQ Topic Exchange를 통해 비동기로 전달되거나 http request를 통해 동기적으로 전달됩니다.

Outbox Pattern

유실 불가 이벤트는 Spring Modulith Outbox 패턴을 적용하고, 유실 허용 이벤트는 Outbox DB 저장 없이 바로 publish합니다.

Quorum & DLQ

모든 큐는 Quorum Queue 기반이며, 재시도 5회 초과 시 DLQ(Dead Letter Queue)로 이동합니다.

서비스 간 통신 방식

연결 목적에 따라 비동기(RabbitMQ)와 동기(HTTP/OpenFeign) 통신을 구분해 사용합니다.

ASync

비동기 통신

주문 ↔ 결제

RabbitMQ(Broker)를 사용하여 비동기적으로 통신합니다.

편딩 ↔ 채팅

RabbitMQ(Broker)를 사용하여 비동기적으로 통신합니다.

채팅 · 주문 · 편딩 ↔ 알림

RabbitMQ(Broker)를 사용하여 비동기적으로 통신합니다.

Sync

동기 통신

인증 ↔ 채팅, Spring Cloud Gateway

Eureka를 통해 ip주소를 찾아 HTTP Request를 사용하여 동기적으로 통신합니다.

인증 ↔ 멤버

OpenFeign를 사용하여 동기적으로 통신합니다.

CI 구조 (Multi-module)



서비스 독립 파이프라인 각 서비스마다 별도의 워크플로우 파일을 구성하여, 변경이 발생한 서비스만 선별적으로 빌드 및 테스트를 실행합니다.



최적화 기법 적용 Sparse Checkout으로 해당 서비스와 의존 모듈만 체크아웃하여 시간을 단축하고, Gradle 의존성 캐시를 활용해 매 빌드 재다운로드를 방지합니다.



Matrix 빌드 전략 변경 감지(detect-changes) Job을 통해 git diff로 변경된 서비스를 추출하고, 해당 서비스 목록을 기반으로 병렬 빌드 및 테스트를 수행합니다.



Jacoco 커버리지 auth, chat, member, payment 등 각 서비스 단위로 커버리지 리포트를 수집하여 품질을 관리합니다.

CD 구조

1 서비스별 독립 배포, 공용 로직은 단일화

서비스마다 CI 워크플로우는 독립적이지만, 배포(CD) 로직은 `cd-common.yml` 하나의 재사용 워크플로우(`workflow_call`)로 통합

2 CI 아티팩트 재사용으로 이중 빌드 제거

CD는 이미지를 새로 빌드하기 위해 소스를 다시 받지 않고, CI가 테스트를 통과시켜 업로드해둔 fat JAR를 `run-id`로 지목해 그대로 내려받아 사용합니다.

3 이미지 태그 = 커밋 SHA로 배포 추적성 확보

Docker 이미지 태그를 `latest`가 아니라 배포를 트리거한 commit SHA로 고정.

4 SSM Run Command 기반 중단 배포

GitHub Actions에서 AWS SSM Run Command로 직접 배포.
`--max-concurrency 1`,
`--max-errors 0` 설정으로 순차 배포·실패 시 확산 중단이 보장되며, 별도 배포 에이전트 설치가 필요 없습니다.

5 비밀 정보의 짧은 생존 주기

DB 계정·API 키는 GitHub Secrets 대신 SSM Parameter Store(SecureString)에서 관리. 배포 스크립트가 대상 EC2 안에서 파라미터를 임시 `env` 파일로 받아 컨테이너에 주입한 뒤, 배포 직후 즉시 삭제합니다.

6 헬스체크 통과 후에만 배포 완료 처리

컨테이너 교체 후 `/actuator/health`를 최대 150초간 폴링해 `status: UP`을 확인한 뒤에야 배포 성공으로 처리. 실패 시 컨테이너 로그 100줄을 자동 덤프해 EC2 접속 없이 원인 파악이 가능합니다.

7 키 없는 인증 (OIDC)

GitHub Actions는 AWS 액세스 키를 저장하지 않고, OIDC로 발급받는 임시 자격 증명을 사용. 신뢰 정책이 특정 레포·브랜치로 제한되어 다른 레포나 브랜치는 이 배포 권한을 빌릴 수 없습니다.

Terraform 인프라

1 Infrastructure as Code로 전체 인프라 관리

VPC부터 EC2, ALB, IAM, DNS, CDN까지 전체 AWS 리소스를 Terraform 코드로 선언.

2 변수 하나로 전체 인프라 On/Off 토크

running 변수로 NAT Gateway, EC2 9대, ALB를 한 번에 stop/start. terraform apply -var="running=false" 한 줄로 시간당 과금 리소스를 내리고, 다시 apply만으로 원복해 비용을 최대 90% 이상 절감합니다.

3 최소 권한 원칙의 3단계 IAM 분리

장기 액세스 키는 로컬 관리자 계정 하나로 한정. GitHub Actions(OIDC 임시 인증)와 EC2(인스턴스 프로파일)는 각각 필요한 권한만 가진 별도 역할로 분리해, 하드코딩된 키가 어디에도 없습니다.

4 리소스 간 자동 의존성 해석

서브넷·보안그룹·IAM 역할·EC2 등을 이름으로 참조해, 하나가 재생성돼도 참조하는 리소스가 자동으로 최신 값을 따라갑니다. ID를 수동으로 복사해 옮기는 작업이 필요 없습니다.

5 네트워크 비용 최적화 (S3 Gateway Endpoint)

프라이빗 서브넷에서 S3(ECR 이미지화 레이어 포함)로 가는 트래픽이 NAT Gateway를 우회하도록 무료 VPC 엔드포인트를 구성. 배포마다 발생하는 이미지 pull 트래픽의 NAT 비용을 없앴습니다.

6 HTTPS 인증서 자동 발급·갱신 (멀티 리전)

ALB용 인증서(서울 리전)와 CloudFront용 인증서(us-east-1)를 각각의 provider로 자동 발급·DNS 검증. 인증서 만료·재발급을 수동으로 신경 쓸 필요가 없습니다.

7 서비스 목록과 파생 리소스의 단일 진실 공급원(SSOT)

EC2 인스턴스 목록(instances 변수) 하나로부터 ECR 리포지토리, EC2 태그, on/off 대상이 자동 파생. 새 서비스 추가 시 목록 한 곳만 수정하면 관련 인프라가 일관되게 따라갑니다.

8 정적 프론트엔드 배포 인프라 (S3 + CloudFront + OAC)

S3 버킷을 완전 비공개로 유지하고 CloudFront OAC로만 접근을 허용. SPA 클라이언트 라우팅을 위한 403/404 → index.html 리다이렉트도 함께 구성했습니다.

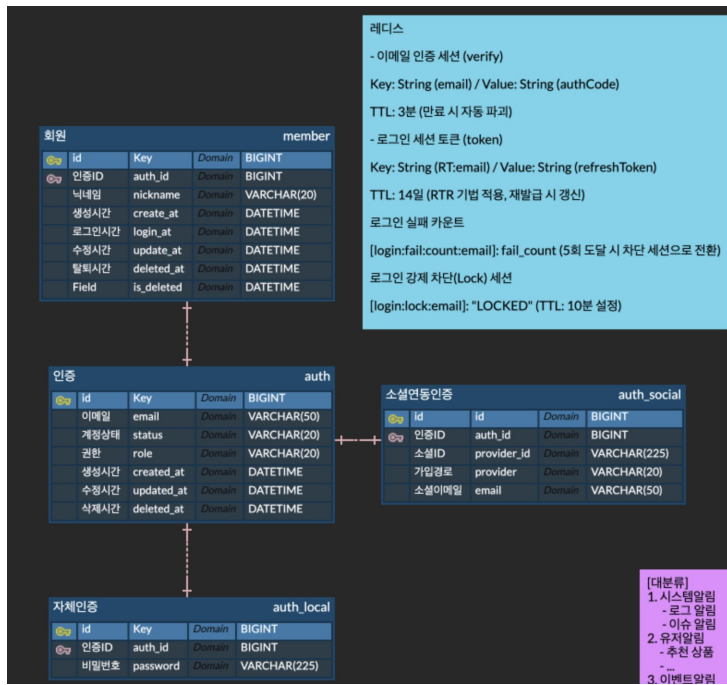


SERVICE 01

인증 · 멤버

회원 인증/인가 구조와 RSA·JWKS, 게이트웨이 중심 JWT 검증 아키텍처를
다룹니다.

ERD



인증 / 회원 분리

- 프로필 데이터를 담은 회원과 비밀번호·계정상태를 담은 인증을 분리했습니다.

레디스 활용

- 이메일 인증코드, refresh 토큰 등 일정 시간 뒤 자동 소멸하는 데이터를 RDB 대신 Redis TTL로 관리합니다.

EndPoint

Authentication 도메인

엔드포인트 목록

method	path	auth required	설명
POST	/api/auths/mail	N	이메일 인증 번호 발송(1)
POST	/api/auths/mailCheck	N	이메일 인증 번호 검증(2)
POST	/api/auths/login	N	로그인 및 토큰 발급(3)
POST	/api/auths/token/refresh	N	refresh 토큰 재발급(5)
POST	/api/auths/password/verify	N	비밀번호 변경 권한 획득(6)
GET	/api/auths/logout	O	로그아웃(7)

Member 도메인

엔드포인트 목록

method	path	auth required	설명
POST	/api/members/signup	N	플랫폼 자체 회원가입(1)
POST	/api/members/social	N	소셜 회원가입 및 로그인(2)
GET	/api/members/info	O	내 정보(이메일, 닉네임) 조회(6)
PATCH	/api/members/info	O	내 정보(닉네임) 수정(7)
DELETE	/api/members/delete	O	회원 탈퇴(10)
PATCH	/api/members/role	O	회원 권한 변경(서포터, 메이커)(11)

Jacoco Coverage

auth-service

Element	Missed Instructions	Cov.	Missed Branches	Cov.
com.bds.auth.domain.entity		96%		92%
com.bds.auth.application		99%		94%
com.bds.auth.infrastructure.security		100%		n/a
Total	6 of 668	99%	2 of 32	93%

gateway-service

Element	Missed Instructions	Cov.	Missed Branches	Cov.
com.dbs.gateway.filter		98%		100%
com.dbs.gateway.config		100%		n/a
com.dbs.gateway.security		100%		n/a
Total	2 of 176	98%	0 of 4	100%

member-service

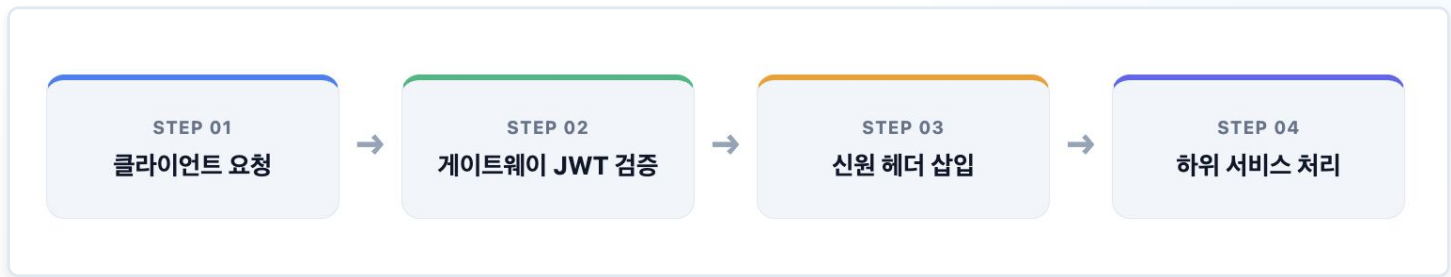
Element	Missed Instructions	Cov.	Missed Branches	Cov.
com.bds.member.application		100%		87%
com.bds.member.domain.entity		100%		90%
Total	0 of 201	100%	3 of 26	88%

entity, service 위주로 테스트 코드 작성

게이트웨이 중심 JWT 검증

마이크로서비스 환경에서 각 서비스가 독립적으로 인증을 처리하면 중복 구현과 검증 불일치가 발생합니다.

해결: 게이트웨이에서 인증을 단일 책임으로 담당하고, 하위 서비스는 이미 검증된 신원만 신뢰하는 구조로 설계



- JWKS 기반 전역 JWT 인증 도입
- 클라이언트는 JWT를 게이트웨이에만 전달
- 게이트웨이가 검증 후 신원 정보를 헤더로 변환
- 하위 서비스는 JWT 구조를 몰라도 됨

내부 요청 위조 방어

보안그룹으로 포트를 막아도, 같은 네트워크 내부에서는 공격자가 **X-User-Id** 헤더를 직접 주입하여 임의 사용자 행세를 할 수 있습니다.

1차 방어 — 인프라

보안그룹으로 서비스 포트 직접 접근을 차단합니다.

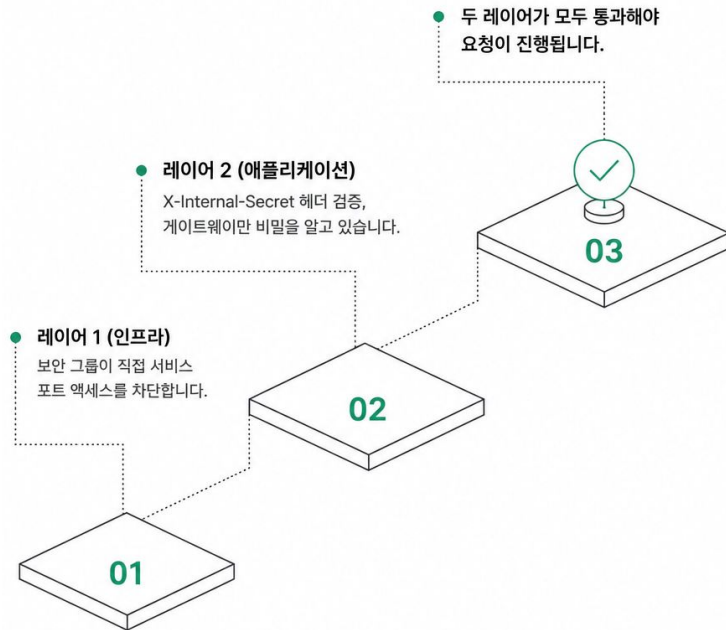
외부에서 내부 서비스로 직접 호출이 불가능합니다.

같은 그룹 내 다른 서비스 인스턴스가 게이트웨이를 거치지 않고 직접 호출하는 것은 막지 못합니다.

2차 방어 — 애플리케이션

게이트웨이만 아는 내부 시크릿(X-Internal-Secret)을 신원 헤더와 함께 요구합니다.

시크릿이 일치하지 않으면 즉시 차단합니다.



인코딩 우회 취약점 수정

```
private boolean isInternalPath(HttpServletRequest request) {  
    return request.getRequestURI().startsWith(INTERNAL_PATH_PREFIX);  
}
```

```
private boolean isInternalPath(HttpServletRequest request) {  
    try {  
        String decodedPath = UriUtils.decode(request.getRequestURI(), StandardCharsets.UTF_8);  
        return decodedPath.startsWith(INTERNAL_PATH_PREFIX);  
    } catch (IllegalArgumentException e) {  
        return true;  
    }  
}.timeout(BLACKLIST_CHECK_TIMEOUT)  
.onErrorReturn(false);  
}
```

문제

인코딩된 경로(/%69nternal/...)로 시크릿 검사를 우회할 수 있었습니다.

필터: 인코딩 전 문자열로 검사 / 라우팅: 디코딩 후 문자열로 판단

→ 기존 불일치가 만든 보안 틈

수정

디코딩 후 비교: 필터도 URL 디코딩을 수행한 후 내부 경로(/internal/)와 비교합니다.

Fail-Closed 처리: 디코딩 실패 시 요청을 차단하고 오류를 반환합니다.

로그아웃 블랙리스트

Redis 블랙리스트는 Auth Service만 접근 가능합니다. 게이트웨이는 매 요청마다 `/internal/auths/blacklist`로 확인합니다.

Fail-Open (현재)

응답 없음 → false → 토큰 통과

리스크: 로그아웃 토큰이 장애 중 일시 허용됩니다.

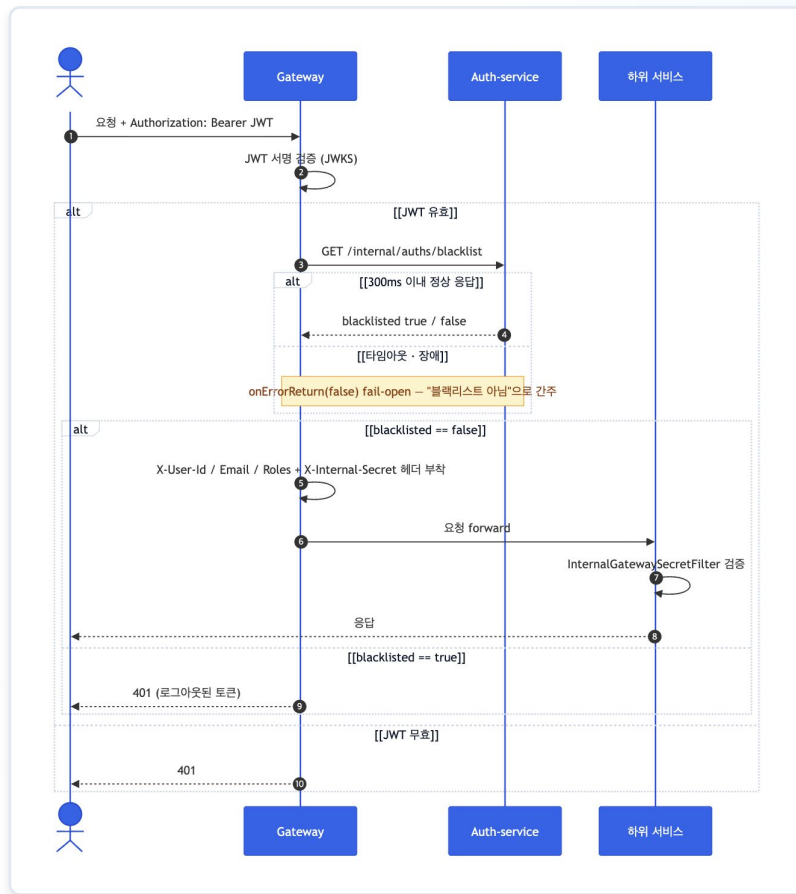
Fail-Closed (대안)

응답 없음 → 차단 → 전체 서비스 마비

Auth Service가 단일 장애점이 됩니다.

```
public Mono<Boolean> isBlacklisted(String accessToken) {  
    return webClient.get()  
        .uri(BLACKLIST_CHECK_PATH)  
        .header(HttpHeaders.AUTHORIZATION, "Bearer " + accessToken)  
        .header(INTERNAL_SECRET_HEADER, internalGatewaySecret)  
        .retrieve()  
        .bodyToMono(Boolean.class)  
        .timeout(BLACKLIST_CHECK_TIMEOUT)  
        .onErrorReturn(false);  
}
```

게이트웨이 흐름





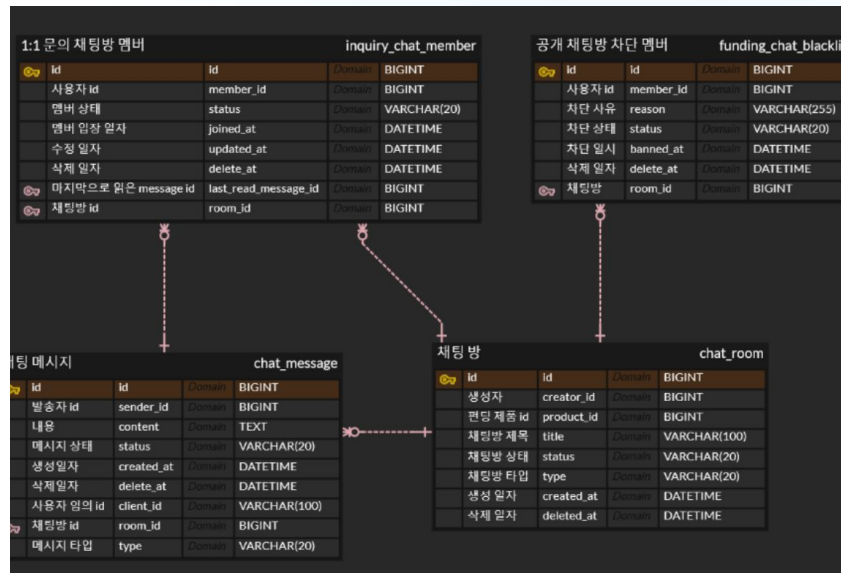
SERVICE 02

채팅

WebSocket(STOMP) 기반 실시간 채팅과 STOMP 인증, 세션·토큰 생명주기를
다룹니다.

ERD

- **채팅방 유형:** 공개 편딩 채팅방과 1:1 문의 채팅방으로 분리되어 운영됩니다.
- **멤버 관리:** 1:1 문의방은 명시적인 멤버 정보를 저장하며, 공개방은 별도 멤버 정보 없이 자유로운 참여를 보장합니다.
- **관리 기능:** 악성 유저 차단을 위한 Blacklist(Ban) 기능은 공개 채팅방에 한정하여 지원합니다.



EndPoint

method	path	auth required	설명
POST	/api/chat/Inquiries	O	1:1 문의 채팅방 생성
GET	/api/chat/Inquiries	O	내 참여 문의 채팅방 목록 조회
GET	/api/chat/Inquiries/{roomId}	O	1:1 문의 채팅방 상세 조회
DELETE	/api/chat/Inquiries/{roomId}/members/me	O	1:1 문의 채팅방 나가기
DELETE	/api/chat/rooms/{roomId}/close	O	공개 채팅방 삭제
POST	/internal/chat/fundings/{productId}	X	펀딩 제품 생성시 공개 채팅방 자동 생성 (시스템)
GET	/api/chat/fundings/{productId}	X	공개 채팅방 조회
POST	/api/chat/fundings/{roomId}/ban	O	공개 채팅방 사용자 BAN
DELETE	/api/chat/fundings/{roomId}/ban/{targetId}	O	공개 채팅방 사용자 BAN 해제
GET	/api/chat/rooms/messages	O	채팅 이력 조회
GET	/api/chat/Inquiries/{roomId}/messages	O	1:1 문의 채팅방 메시지 조회
GET	/api/chat/fundings/{roomId}/messages	O	공개 채팅방 메시지 조회
DELETE	/api/chat/messages/{messageId}	O	메시지 삭제(soft delete)
WS	/ws/chat	X	Websocket 연결
SUB	/topic/chat.room.{roomId}	O/X	채팅방 구독 (메시지/이벤트)
SUB	/topic/chat.room.{roomId}.read	O/X	채팅방 읽음 이벤트 구독
UNSUB	subscription-id	O/X	채팅방 구독 취소
PUB	/app/chat/send/{roomId}	O	메시지 전송
PUB	/app/chat/read/{roomId}	O	읽음 상태 갱신
PUB	/app/auth/refresh	O	엑세스 토큰 갱신

Jacoco Coverage

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed	Cxty	Missed	Lines	Missed	Methods	Missed	Classes
com.bds.chat.domain.shared		59%		52%	24	54	17	66	7	30	0	6
com.bds.chat.infrastructure.persistence.message		80%		73%	10	32	15	75	4	19	0	2
com.bds.chat.infrastructure.session		92%		74%	27	93	19	193	5	47	0	10
com.bds.chat.application.chatRoom.service		92%		78%	7	51	8	127	2	37	0	2
com.bds.chat.presentation		73%		80%	5	15	11	43	3	10	0	1
com.bds.chat.application.chatRoom.dto		88%		100%	2	21	7	53	2	19	1	11
com.bds.chat.application.message.service		94%		75%	11	45	5	93	1	23	0	1
com.bds.chat.infrastructure.security		90%		55%	9	25	6	63	1	16	0	3
com.bds.chat.presentation.stomp		89%		83%	1	13	5	46	0	10	0	7
com.bds.chat.presentation.stomp.dto		75%		n/a	1	4	1	5	1	4	1	2
com.bds.chat.infrastructure.config		97%		50%	3	51	2	136	1	49	0	9
com.bds.chat.domain.chatRoom		94%		87%	1	13	1	35	0	9	0	3
com.bds.chat.common		92%		n/a	2	13	4	28	2	13	1	5
com.bds.chat.infrastructure.persistence.member		96%		100%	0	19	2	55	0	17	0	2
com.bds.chat		37%		n/a	1	2	2	3	1	2	0	1
com.bds.chat.domain.member		99%		91%	2	27	0	54	0	15	0	3
com.bds.chat.infrastructure.messaging		98%		100%	1	15	1	40	1	12	0	4
com.bds.chat.application.blackList.service		100%		100%	0	11	0	29	0	5	0	1
com.bds.chat.infrastructure.persistence.chatroom		100%		100%	0	12	0	38	0	9	0	2
com.bds.chat.domain.message		100%		100%	0	13	0	30	0	10	0	4
com.bds.chat.infrastructure.persistence.blacklist		100%		100%	0	8	0	27	0	6	0	2
com.bds.chat.domain.blackList		100%		100%	0	8	0	23	0	6	0	2
com.bds.chat.application.message.dto		100%		100%	0	7	0	12	0	6	0	4
com.bds.chat.presentation.chatRoom		100%		n/a	0	8	0	13	0	8	0	2
com.bds.chat.application.member.service		100%		n/a	0	4	0	12	0	4	0	1
com.bds.chat.presentation.message		100%		n/a	0	4	0	4	0	4	0	1
com.bds.chat.application.member.dto		100%		n/a	0	2	0	5	0	2	0	1
com.bds.chat.presentation.blackList		100%		n/a	0	2	0	4	0	2	0	1
com.bds.chat.application.blackList.dto		100%		n/a	0	2	0	2	0	2	0	2
Total	543 of 6,289	91%	86 of 353	75%	107	574	106	1,314	31	396	3	95

테스트 현황

64개 클래스 **362**개 테스트 케이스

GitHub Actions CI에서 Jacoco 커버리지 리포트와 함께 자동 실행됩니다.

단위 테스트

38개 클래스

Domain · Application · Persistence · Presentation ·
Session/Messaging 레이어, Mockito 기반

통합 테스트

TestContainers

PostgreSQL + RabbitMQ 기반. CRUD, 동시성(중복 방
생성·중복 메시지·중복 멤버십), 멍등성 확인

MSA RabbitMQ 이벤트 수신

FUNDING_START/SUCCESS/FAIL

수신 후 DB 반영 확인, 재전달 시 멍등성(DLQ 미적재) 확인,
queue 재시도(지수 백오프 3회) 및 DLQ 적재 확인

MSA RabbitMQ 이벤트 발행

ChatSendEvent

문의방 메시지 전송 시 chat.exchange 발행 확인
(notification-service 연계), 편당방은 미발행 확인

E2E WebSocket

WireMock JWKS + TestContainers

JWT 인증, 구독 권한(FUNDING 개방·INQUIRY 멤버십 체크),
메시지 송수신, 비인증 세션 거부

채팅 기술 스택



실시간 통신

WebSocket + STOMP으로 실시간
메시지 송수신



메시지 브로커

RabbitMQ 4 + STOMP
플러그인으로
외부 브로커 릴레이로 사용합니다.

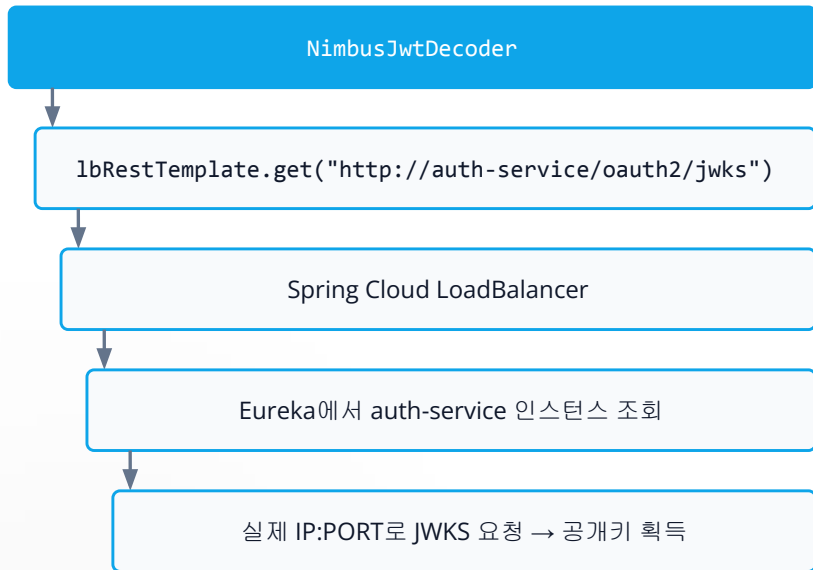


데이터 베이스

PostgreSQL과 Spring Data JPA를
통해 영구 저장소를 구성하며,
Flyway로 스키마 버전을
체계적으로 관리합니다.

Eureka 연동

SecurityConfig의 JwtDecoder 빈에 @LoadBalanced RestTemplate을 주입합니다.



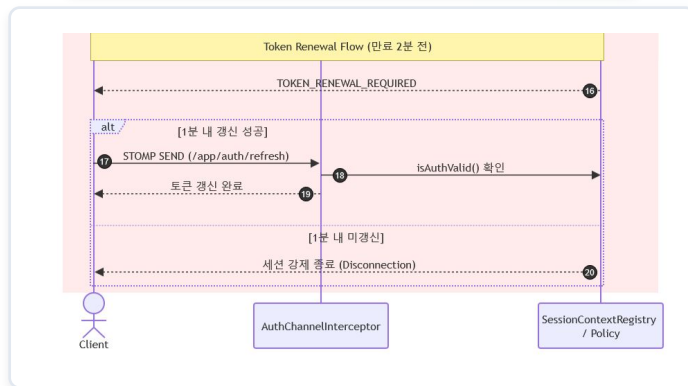
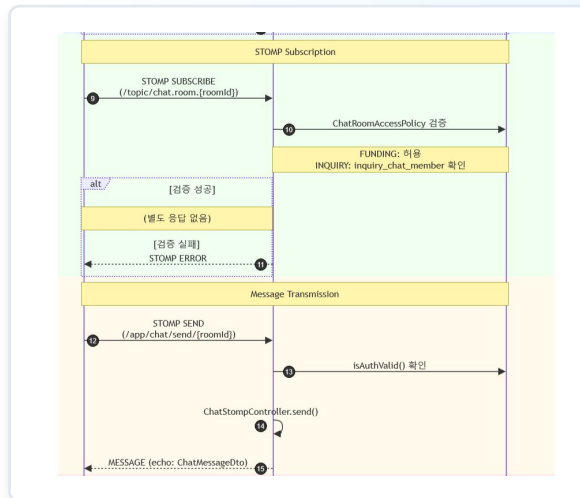
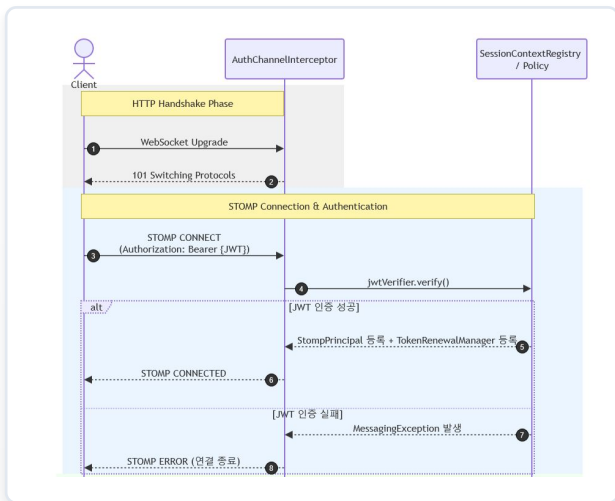
@LoadBalanced가 붙은 RestTemplate은 http://서비스명/... 형태의 URL을 Spring Cloud LoadBalancer가 가로채 Eureka 레지스트리에서 실제 인스턴스로 라우팅합니다.

NimbusJwtDecoder가 내부적으로 이 RestTemplate을 이용해 JWKS를 fetch하므로, auth-service의 실제 IP를 하드코딩할 필요가 없습니다.

```
// SecurityConfig.java
@Bean
@LoadBalanced
public RestTemplate lbRestTemplate() {
    return new RestTemplate();
}

@Bean
@ConditionalOnProperty(name = "app.auth.dev-mode",
    havingValue = "false", matchIfMissing = true)
public JwtDecoder jwtDecoder(
    @Value("${app.auth.jwks-uri}") String jwksUri,
    @Value("${app.auth.issuer}") String issuer,
    RestTemplate lbRestTemplate) {
    NimbusJwtDecoder decoder = NimbusJwtDecoder
        .withJwkSetUri(jwksUri)
        .restOperations(lbRestTemplate) // ← @LoadBalanced 주입
    if (!issuer.isBlank()) {
        decoder.setJwtValidator(
            JwtValidators.createDefaultWithIssuer(issuer));
    }
    return decoder;
}
```

인증 흐름 - STOMP 채널 레벨



Requeue 재시도 및 DLQ 설정

DualBrokerConfig의 msaListenerContainerFactory에 stateless retry를 설정했습니다.

```
f.setDefaultRequeueRejected(false);  
// reject 시 재큐 대신 DLQ로  
f.setAdviceChain(RetryInterceptorBuilder.stateless()  
    .maxRetries(3)  
    .configureRetryPolicy(b -> b  
        .delay(Duration.ofSeconds(1))  
        .multiplier(2.0)  
        .maxDelay(Duration.ofSeconds(10)))  
    .build());
```

큐 구조 (BdsQueues.workQueue)

funding.exchange (topic)

chat.funding-created.queue — 메인 소비 큐 (quorum, DLX 설정)

x-dead-letter-exchange: dlx.exchange · x-dead-letter-routing-key: ...dead ·
x-delivery-limit: 5 (최대 5회 재전달 후 DLQ로)

dlx.exchange (direct)

chat.funding-created.queue.dlq — 데드 레터 큐 (quorum)

재시도 흐름

일시적 오류 발생

Spring Retry (최대 3회, 1s → 2s → 4s 지수 백오프)

성공 → ack / 3회 모두 실패 → nack (requeue=false)

RabbitMQ DLX 라우팅

chat.funding-created.queue.dlq 적재



SERVICE 03

주문

주문 도메인 설계와 Funding·Reward 처리, RabbitMQ 기반 이벤트 흐름을
다룹니다.

ERD



- 주문 서비스에 Funding & Reward 컨텍스트를 포함합니다.
- order_reward로 다대다 분리, 주문 시점 금액을 스냅샷으로 보관합니다.
- event_publication: Outbox 패턴을 적용해 이벤트 유실을 방지하며, 도메인 테이블과 독립적으로 관리합니다.

EndPoint

Order 도메인

method	path	auth	설명
POST	/api/orders/billing	O	주문서 생성 (정보 조회, DB X)
POST	/api/orders	O	주문 생성 (재고 차감 + 결제 호출)
GET	/api/orders	O	내 주문 목록 조회
GET	/api/orders/{orderId}	O	주문 상세 조회
PATCH	/api/orders/{orderId}/cancel	O	사용자 주문 취소

Funding 도메인

method	path	auth	설명
GET	/api/fundings	X	펀딩 목록 조회 (상태 필터링 가능)
GET	/api/fundings/{fundingId}	X	펀딩 상세 및 리워드 목록 조회
POST	/api/fundings	O	펀딩 생성 (리워드 동시 등록, 메이커 전용)

Jacoco Coverage

order-service

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed	Cxty	Missed	Lines	Missed	Methods	Missed	Classes
com.bds.order.infrastructure.scheduler		91%		97%	2	59	13	196	0	23	0	4
com.bds.order.application		93%		84%	10	71	16	223	1	37	0	4
com.bds.order.infrastructure.funding		98%		88%	2	31	2	97	1	25	0	6
com.bds.order.domain.order		98%		88%	6	42	1	46	0	14	0	3
com.bds.order.presentation.dto		100%		100%	0	26	0	86	0	25	0	16
com.bds.order.global.exception		100%		n/a	0	23	0	80	0	23	0	4
com.bds.order.infrastructure.order		100%		100%	0	17	0	53	0	15	0	5
com.bds.order.domain.funding		100%		100%	0	11	0	23	0	8	0	3
com.bds.order.presentation.controller		100%		n/a	0	8	0	15	0	8	0	2
com.bds.order.infrastructure.config		100%		n/a	0	13	0	19	0	13	0	4
com.bds.order.infrastructure.reward		100%		n/a	0	6	0	21	0	6	0	2
com.bds.order.domain.reward		100%		100%	0	6	0	7	0	4	0	2
com.bds.order.infrastructure.orderReward		100%		n/a	0	6	0	12	0	6	0	4
com.bds.order.infrastructure.messaging.consumer		100%		n/a	0	4	0	10	0	4	0	1
com.bds.order.infrastructure.messaging.publisher		100%		n/a	0	7	0	12	0	7	0	2
com.bds.order.domain.orderReward		100%		n/a	0	2	0	2	0	2	0	1
com.bds.order.infrastructure.common		100%		n/a	0	1	0	1	0	1	0	1
Total	167 of 4,642	96%	19 of 212	91%	20	333	32	903	2	221	0	64

목표 커버리지 Line, Branch 70% 달성 (total 347)

Message Queue

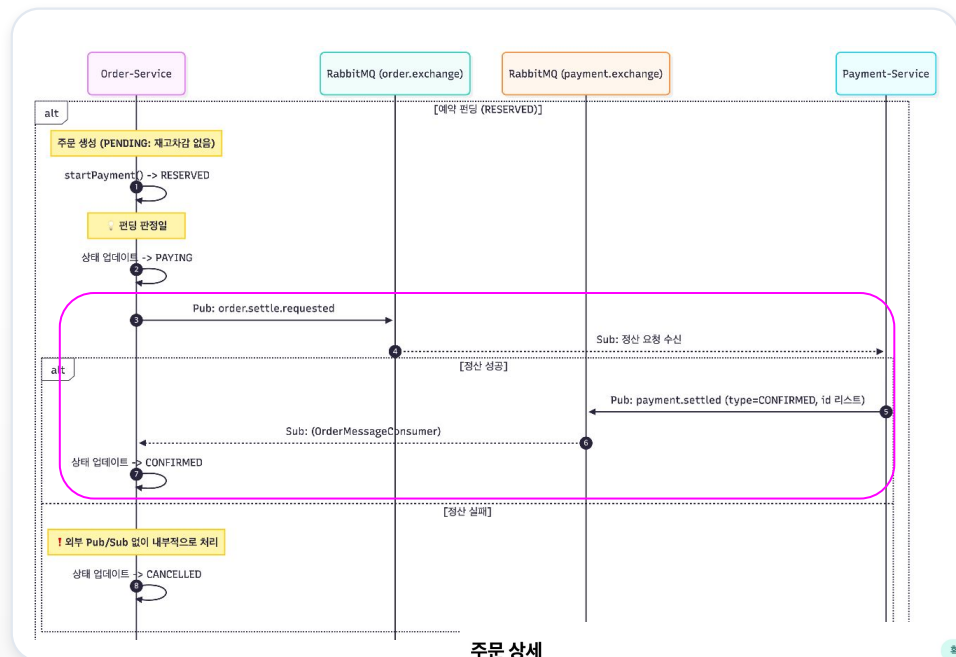
Order.exchange 발행

To	Routing key	Arguments	
notification.order-status.queue	order.status		Unbind
payment.order.pay.queue	order.pay.requested		Unbind
payment.order.refund.queue	order.refund.requested		Unbind
payment.order.settlement.queue	order.settle.requested		Unbind

Order 수신 큐

msa	order.payment-cancelled.queue	quorum
msa	order.payment-cancelled.queue.dlq	quorum
msa	order.payment-paid.queue	quorum
msa	order.payment-paid.queue.dlq	quorum
msa	order.payment-settled.queue	quorum
msa	order.payment-settled.queue.dlq	quorum

1. 예약 펀딩 - 펀딩 성공 및 정산

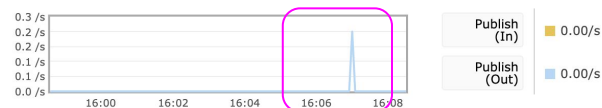


1. CONFIRMED 정산 요청 (메시지 발행)

Exchange: order.exchange

Overview

Message rates last ten minutes ?



2. CONFIRMED 응답 및 상태변경 (메시지 수신)

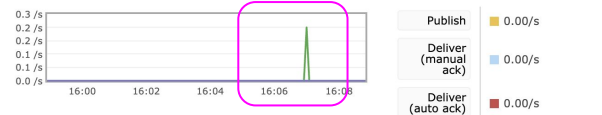
Queue order.payment-settled.queue

Overview

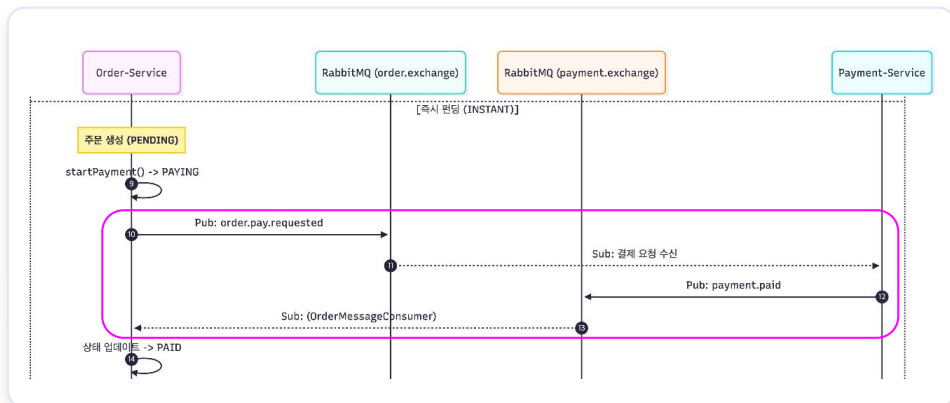
Queued messages last ten minutes ?



Message rates last ten minutes ?



2. 즉시 펀딩 - 결제 완료



주문 상세

결제 완료

✓ 결제 완료

결제가 완료되었습니다. 펀딩 종료 후 정산이 진행됩니다.

주문번호

ORD-452A9094226E

펀딩

스마트 이어플러그

주문일시

2026. 7. 27. 오후 3:58:26

1. PAID 요청(메시지 발행)

Exchange: order.exchange

Overview

Message rates last ten minutes ?

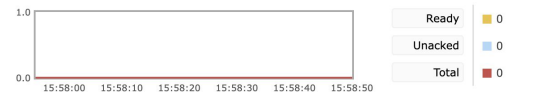


2. PAID 응답 및 상태변경(메시지 수신)

Queue order.payment-paid.queue

Overview

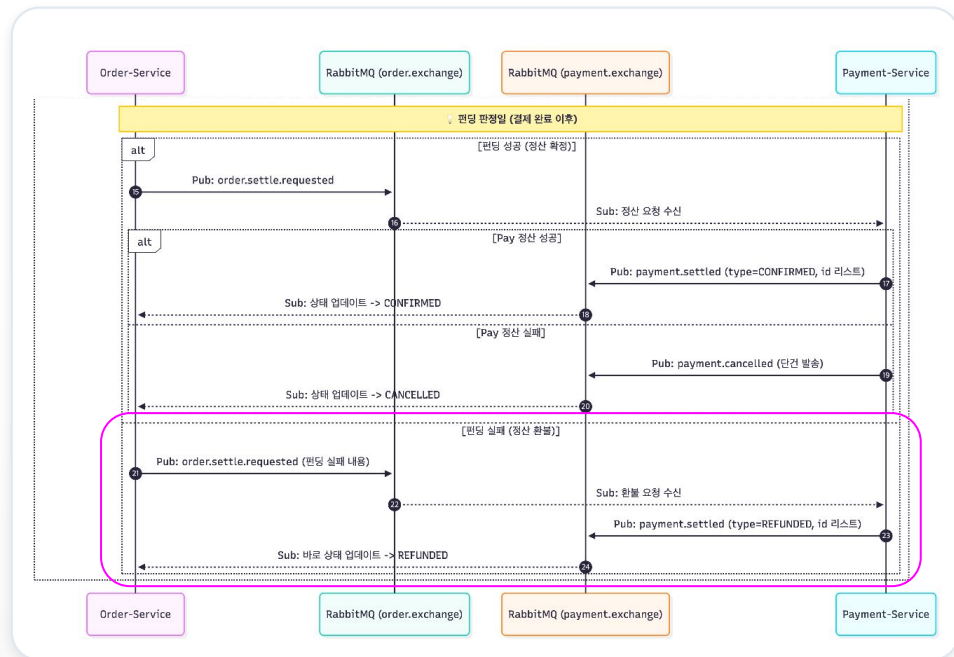
Queued messages last minute ?



Message rates last minute ?



3. 즉시 펀딩 - 펀딩 실패 및 환불



1. REFUND 요청(메시지 발행)

Exchange: order.exchange

Overview

Message rates last ten minutes ?



2. REFUNDED 수신 및 프론트 결과

내 주문 내역

즉시펀딩실패/환불 테스트 ORD-003128CAB736 2026. 7. 27. 오후 4:28:34	환불 완료 33,000원
즉시펀딩실패/환불 테스트 ORD-92025AB6F9A5 2026. 7. 27. 오후 4:28:24	환불 완료 48,000원
즉시펀딩실패/환불 테스트 ORD-4F2FE4C21374 2026. 7. 27. 오후 4:28:19	환불 완료 33,000원
즉시펀딩실패/환불 테스트 ORD-750000000000 2026. 7. 27. 오후 4:28:14	환불 완료 15,000원



SERVICE 04

결제

결제 서비스 설계와 Transactional Outbox 기반 이벤트 처리 구조를
다룹니다.

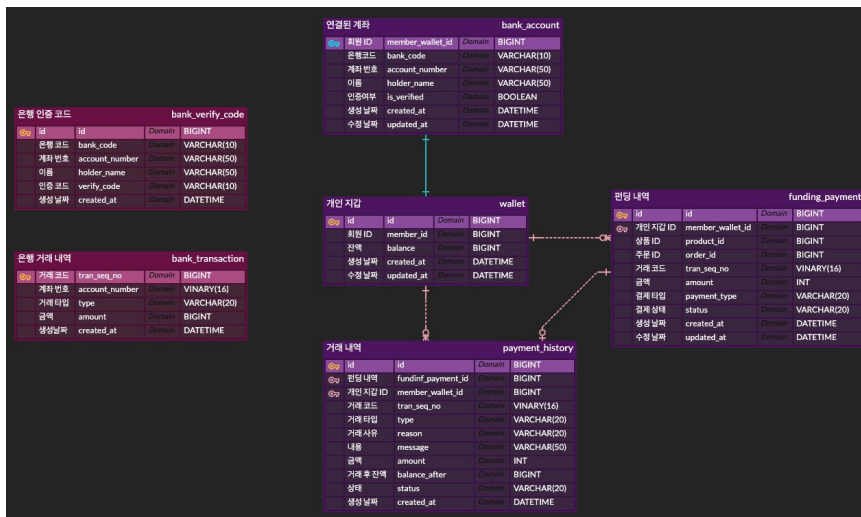
ERD

가상금융망 (Mock)

- 계좌연동을 위한 은행망
- 페이 충전, 환전을 담당

결제 서비스

- 자체 페이 서비스, 인당 1개의 계좌
- 주문 도메인에서 진행되는 펀딩 결제 담당



EndPoint

payment 도메인 엔드포인트 목록

method	path	auth required	설명
GET	/api/payment/wallet	O	월렛 잔액 조회
POST	/api/payment/accounts	O	계좌 등록
POST	/api/payment/accounts/verify	O	1원 인증
POST	/api/payment/deposit	O	페이 충전
POST	/api/payment/withdraw	O	페이 출금
GET	/api/payment/history	O	개인 월렛 거래 내역 조회
POST	/api/payment/funding	X	펀딩 결제 (RabbitMQ 로 구현 예정)
POST	/api/payment/funding/settlement	X	펀딩 정산 요청 (RabbitMQ 로 구현 예정)
POST	/api/payment/funding/batch-refund	X	펀딩 배치 환불 요청 (RabbitMQ 로 구현 예정)
POST	/api/payment/refund	O	수동 환불 (RabbitMQ 로 구현 예정)

가상 금융망 엔드포인트 목록

method	path	auth required	설명
POST	/api/banks/accounts	X	계좌 실명 조회 + 1원 송금
POST	/api/banks/accounts/verify	X	인증 코드 조회
POST	/api/banks/withdraw	X	계좌 출금 (페이 충전)
POST	/api/banks/deposit	X	계좌 입금 (원화 환불)
GET	/api/banks/transactions/{tranSeqNo}	X	거래 코드 조회

Jacoco Coverage

payment-service

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed	Cxty	Missed	Lines	Missed	Methods	Missed	Classes
com.bds.payment.payment.infrastructure.external		3%		0%	5	6	38	39	4	5	0	1
com.bds.payment.payment.global.exception		80%		0%	9	13	25	71	7	11	0	3
com.bds.payment.payment.presentation.response		60%		n/a	6	17	13	33	6	17	3	10
com.bds.payment.payment.presentation.controller		0%		n/a	7	7	7	7	7	7	3	3
com.bds.payment.payment.application.funding		96%		93%	5	65	8	219	1	35	0	11
com.bds.payment.payment.infrastructure.persistence.fundingPayment		82%		n/a	2	11	3	41	2	11	0	3
com.bds.payment.payment.domain.fundingPayment		75%		62%	5	15	8	31	1	7	0	1
com.bds.payment.bank.presentation.controller		0%		n/a	5	5	5	5	5	5	1	1
com.bds.payment.payment.infrastructure.external.response		28%		n/a	3	4	3	4	3	4	2	3
com.bds.payment.payment.infrastructure.persistence.paymentHistory		81%		n/a	2	7	2	32	2	7	0	3
com.bds.payment.payment.presentation.request		86%		n/a	1	8	1	8	1	8	1	8
com.bds.payment.payment.infrastructure.persistence.wallet		91%		83%	2	12	1	26	1	9	0	3
com.bds.payment.payment.infrastructure.persistence.account		91%		100%	2	11	3	29	2	8	0	3
com.bds.payment.payment.application.funding.consumer		100%		92%	1	15	0	60	0	7	0	3
com.bds.payment.payment.application.payment		100%		100%	0	12	0	36	0	9	0	2
com.bds.payment.payment.domain.common		100%		n/a	0	7	0	18	0	7	0	7
com.bds.payment.payment.application.wallet		100%		100%	0	15	0	27	0	13	0	1
com.bds.payment.bank.application		100%		100%	0	12	0	22	0	9	0	1
com.bds.payment.payment.application.accounts		100%		100%	0	8	0	23	0	5	0	1
com.bds.payment.payment.infrastructure.config		100%		100%	0	12	0	21	0	11	0	4
com.bds.payment.bank.infrastructure.persistence.bankVerifyCode		100%		n/a	0	6	0	21	0	6	0	2
com.bds.payment.bank.infrastructure.persistence.BankTransaction		100%		n/a	0	5	0	19	0	5	0	2
com.bds.payment.payment.infrastructure.external.request		100%		n/a	0	6	0	6	0	6	0	3
com.bds.payment.payment.domain.account		100%		n/a	0	3	0	19	0	3	0	1
com.bds.payment.bank.presentation.response		100%		n/a	0	6	0	6	0	6	0	4
com.bds.payment.payment.domain.wallet		100%		100%	0	4	0	12	0	3	0	1
com.bds.payment.bank.presentation.request		100%		n/a	0	3	0	3	0	3	0	3
com.bds.payment.payment.domain.paymentHistory		100%		n/a	0	1	0	11	0	1	0	1
com.bds.payment.bank.domain.bankVerifyCode		100%		n/a	0	1	0	8	0	1	0	1
com.bds.payment.bank.domain.bankTransaction		100%		n/a	0	1	0	7	0	1	0	1
com.bds.payment.bank.domain.common		100%		n/a	0	1	0	2	0	1	0	1
Total	546 of 4,145	86%	17 of 133	87%	55	299	117	866	42	231	10	92

각 서비스 기준 커버리지 90%

단건 결제

```
public FundingPaymentResponseDto funding(FundingPaymentRequestDto dto) {  
    ...  
    try {  
        PaymentResult result = paymentProcessor.process(ctx);  
  
        if (result instanceof PaymentResult.Success) {  
            eventPublisher.publishOrderPaid(dto.orderId());  
        }  
  
        return FundingPaymentResponseDto.from(result.funding(), dto.memberId());  
    } catch (BusinessException e) {  
        if (isUserFault(e)) {  
            failureHandler.handleUserFailure(ctx, e);  
            ...  
            return FundingPaymentResponseDto.from(failed, dto.memberId());  
        }  
        failureHandler.handleSystemError(dto.orderId());  
        throw e;  
    } catch (Exception e) {  
        log.error("Unexpected error during funding. orderId={}", dto.orderId(), e);  
        failureHandler.handleSystemError(dto.orderId());  
        throw new BusinessException(ErrorCode.PAYMENT_SERVER_ERROR);  
    }  
}
```

결제 결과를 타입으로 명시적 분기

성공

이벤트 발행

정상 처리 결과를 이벤트로 발행합니다.

실패 (유저 책임)

재시도해도 동일 실패

그 자리에서 확정 처리합니다.

실패 (서비스 책임)

재시도하면 성공 가능

다르게 처리(재시도 대상으로 분류)합니다.

판매자 정산

정산 배치는 최대 500건씩 나뉘어 거의 동시에 들어올 수 있습니다.

FOR UPDATE 락으로 아직 크레딧 안 된 건만 조회하고, 이미 처리된 배치는 재시도해도 빈 결과를 받아 조용히 스킵됩니다.

건별 UPDATE 대신 벌크 쿼리 1번으로 처리해 쿼리 수도 최소화했습니다.

```
@Transactional(propagation = Propagation.REQUIRES_NEW)
public void creditCreatorForBatch(Long creatorMemberId, Long productId, String message) {
    List<FundingPayment> uncredited = fundingPaymentRepository
        .findUncreditedForUpdate(productId, FundingPaymentStatus.CONFIRMED);

    if (uncredited.isEmpty()) {
        return;
    }

    long total = uncredited.stream().mapToLong(FundingPayment::getAmount).sum();
    Wallet creatorWallet = walletService.charge(creatorMemberId, total);
    ...
    fundingPaymentRepository.updateCreditedAtBulk(ids, now);
    ...
}
```

```
public interface FundingHistoryJpaRepository extends JpaRepository<FundingPaymentJpaEntity, Long> {
    ...

    @Lock(lockModeType = PessimisticWrite)
    @Query("SELECT fp FROM FundingPaymentJpaEntity fp WHERE fp.productId = :productId AND fp.status = :status AND fp.creditedAt IS NULL")
    List<FundingPaymentJpaEntity> findUncreditedForUpdate(
        @Param("productId") Long productId,
        @Param("status") FundingPaymentStatus status
    );

    @Modifying(clearAutomatically = true)
    @Query("UPDATE FundingPaymentJpaEntity fp SET fp.creditedAt = :now WHERE fp.id IN :ids")
    int updateCreditedAtBulk(@Param("ids") List<Long> ids, @Param("now") LocalDateTime now);
}
```



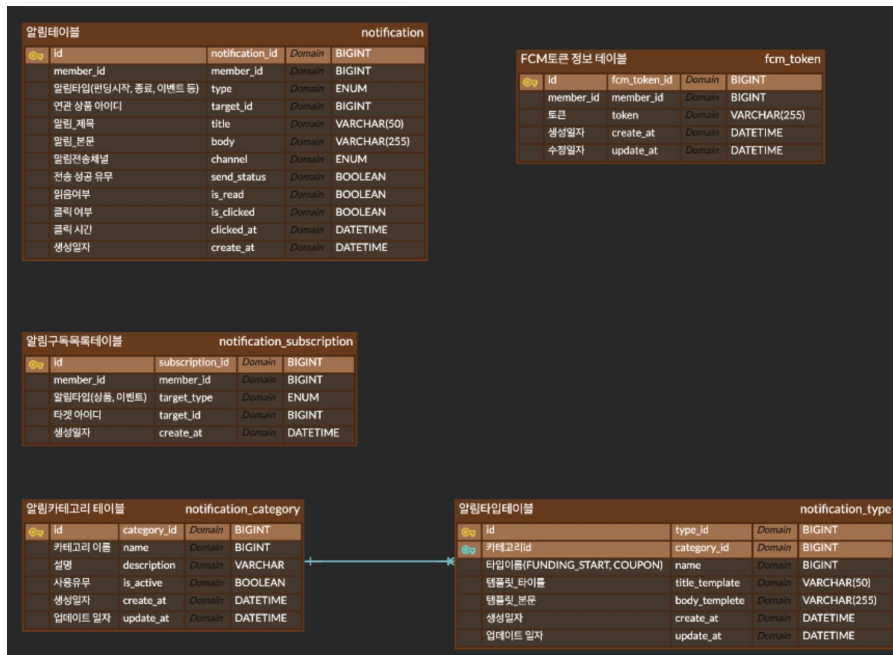
SERVICE 05

알림

알림 도메인 설계와 공용 알림 테이블, SSE / FCM Fallback 전송 구조를
다룹니다.

ERD

- FUNDING 상품이나 프로모션 이벤트의 알림 구독 정보를 저장할 Notification Subscription 테이블
- Funding, Order 등의 상태에 따라 생성되는 Notification 테이블
- 이후 외부 알림을 위한 FCM 토큰 저장 테이블
- 관리자 알림 기능을 위한 알림 타입, 알림 카테고리 테이블로 구성



EndPoint

method	path	auth required	설명
GET	/api/notifications/connect	O	SSE 연결 (알림 구독)
GET	/api/notifications	O	알림 목록 조회 + 전체 읽음 처리
GET	/api/notifications/unread-count	O	읽지 않은 알림 수 조회
POST	/api/notifications/subscriptions	O	알림 구독 등록
DELETE	/api/notifications/subscriptions	O	알림 구독 해지
POST	/api/notifications/fcm-token	O	FCM 토큰 저장
DELETE	/api/notifications/fcm-token	O	FCM 토큰 삭제

Jacoco Coverage

NotificationService

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed	Cxty	Missed	Lines	Missed	Methods
● connect(Long)		100%		n/a	0	1	0	10	0	1
● createFundingNotification(FundingNotificationCommandDto)		83%		75%	2	7	2	15	0	1
● createNotification(Long, NotificationType, String, String, String, NotificationChannel)		100%		n/a	0	1	0	4	0	1
● createOrderNotification(OrderNotificationMessageDto)		89%		83%	1	5	1	10	0	1
● getNotifications(Long, Pageable)		100%		n/a	0	1	0	7	0	1
● getUnreadCount(Long)		100%		n/a	0	1	0	2	0	1
● lambda\$createFundingNotification\$0(FundingNotificationCommandDto, String, String, Long)		100%		n/a	0	1	0	2	0	1
● lambda\$unsubscribe\$0()		100%		n/a	0	1	0	1	0	1
● subscribe(Long, SubscriptionTargetType, Long)		83%		100%	0	2	2	8	0	1
● unsubscribe(Long, SubscriptionTargetType, Long)		100%		n/a	0	1	0	5	0	1
Total	21 of 265	92%	3 of 16	81%	3	21	5	63	0	10

Notification Service 위주로 테스트 커버리지를 확대했으며, 전체 코드 커버리지는 80% 이상입니다.

알림

알림 전송 방식 이벤트 핸들러 방식으로 수정

알림을 생성하는 지점마다 **SseEmitterManager**가 연결돼 있는지 확인하고, 없으면 **FCM**으로 보내는 **if-else**가 반복됐다.

지금은 두 곳뿐이지만 채널이 하나 늘 때마다 이 분기를 모든 호출부에서 다시 수정해야 하는 구조였다.

NotificationEventListener

알림 생성 이벤트를 받아 SSE 연결 여부를 확인하고, 없으면 FCM 메시지를 직접 호출.

NotificationService

채팅 알림 발송에서도 동일한 SSE/FCM 분기가 별도로 한 번 더 반복됨.

● 이전 - 호출부가 채널을 직접 판단

```
if (sseEmitterManager.exist(memberId)) {  
    sseEmitterManager.send(  
        memberId, "notification", payload);  
} else {  
    // TODO: FCM Fallback 예정  
    // → 채널이 늘 때마다 이 블록을  
    // 호출부마다 다시 고쳐야 함  
}
```

이벤트 핸들러 기반 전송 구조 (1/2)

호출부는 "채널이 무엇이냐"를 몰라도 되게 만들고, 채널 선택 책임을 한 곳(디스패처)으로 옮깁니다.

BEFORE

NOTIFICATIONEVENTLISTENER

```
if (sseEmitterManager.exist(memberId)) {
    sseEmitterManager.send(
        memberId, "notification", payload
    );
} else {
    fcmPushManager.send(
        memberId, type, title, body
    );
}
// 채널이 늘어날수록 분기가 계속 늘어난다
```

AFTER

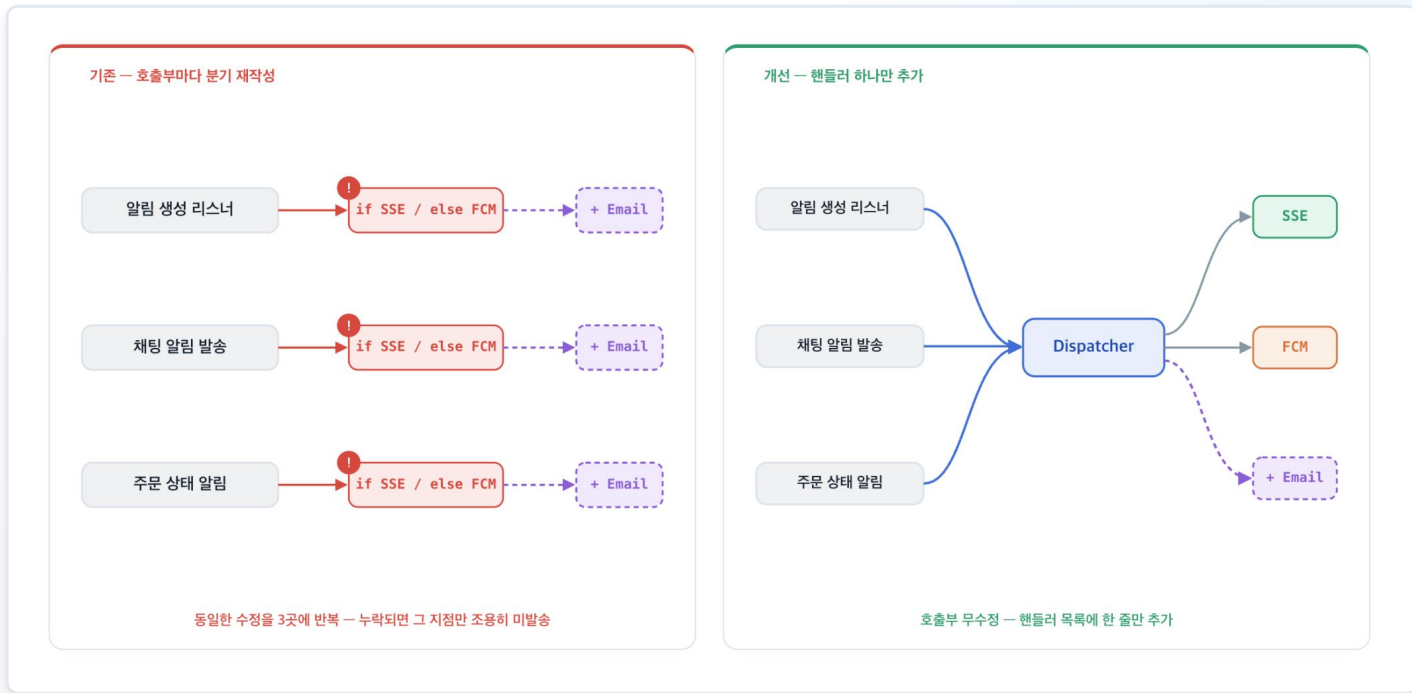
NOTIFICATIONDISPATCHER

```
for (handler : handlers) {
    if (handler.supports(memberId)) {
        handler.handle(memberId, payload);
        return;
    }
}
// 어떤 채널로 보낼지는 각 Handler가 스스로 판단
// Dispatcher는 "누군가 처리한다"는 사실만 안다
```

- 알림 전송 방식을 이벤트 핸들러 방식으로 수정했습니다.
- 실제 알림을 보내는 함수를 Dispatcher로 호출하고, Dispatcher가 가용한 알림 Handler를 호출해 실제 알림으로 전달합니다.

이벤트 핸들러 기반 전송 구조 (2/2)

호출부는 "채널이 무엇이나"를 몰라도 되게 만들고, 채널 선택 책임을 한 곳(디스패처)으로 옮깁니다.



- 알림 수단이 변경되더라도 실제 Handler만 수정하면 일괄적으로 알림 방식을 바꿔서 보낼 수 있습니다.

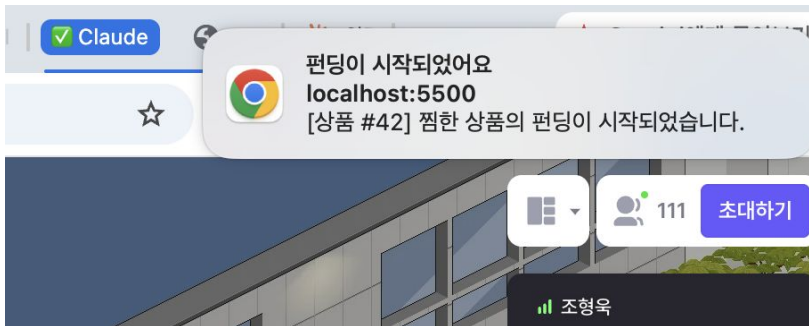
SSE에서 FCM 단일 방식으로 전환

문제

SSE는 사용자가 브라우저에 존재하는 동안만 알림을 보낼 수 있습니다. 브라우저에 없을 때도 중요한 알림이나 구매에 도움이 될 알림은 반드시 전달되어야 합니다.

해결

SSE와 FCM을 하이브리드로 두는 대신, 브라우저 탭에 없을 때는 FCM 기반 브라우저 알림으로 단일화해 구조를 단순하게 만들고 안정성을 높였습니다.



브라우저 탭에 없을 때 — 브라우저 알림

FCM 토큰 발급 테스트

발급된 토큰 (콘솔에도 출력됨):
ckT41n_9350ZPah-
x035g11APM0135dL7YAN0k3sHLTh1rLUzPH6ngYMAnek2M3qb0K0Z91xxMWq1cm06sa33k18JFng25ht
B5XX8ylLk0u4AaX3panfp27PEr-Yaw7woE1j4w5Kkj68

편딩이 시작되었습니다
(상품 #42) 찜한 상품이 편딩이 시작되었습니다.

FCM으로 발송한 Toast 알림

Repository & 배포

프로젝트 소스 코드와 실제 서비스 배포 링크입니다.



Frontend

github.com/KT-Cloud-2-BDS/bds_frontend



Backend

github.com/KT-Cloud-2-BDS/bds_backend



서비스 배포

www.jubg.shop



질문 및 답변

궁금하신 점을 자유롭게 질문해 주세요



감사합니다

끝까지 함께해 주셔서 감사합니다